

TECHNICAL OVERVIEW

Snort 소스 설치 메커니즘

Linux Build System의 이해:
Configuration, Compilation, Installation의 내부 동작

왜 소스 설치가 필수적인가? (Architecture Mismatch)

Ubuntu Repository의 문제점

❌ 버전 파편화 (Fragmentation):

Ubuntu 저장소의 Snort는 안정성을 위해 구버전(v2.9.x)에 머물러 있습니다. 최신 보안 룰셋(Ruleset)은 최신 엔진을 요구하는 경우가 많습니다.

❌ 바이너리 종속성:

`apt install` 로 설치된 바이너리는 미리 컴파일된 상태이므로, 시스템의 특정 라이브러리 버전과 강하게 결합되어 있어 수정이 불가능합니다.

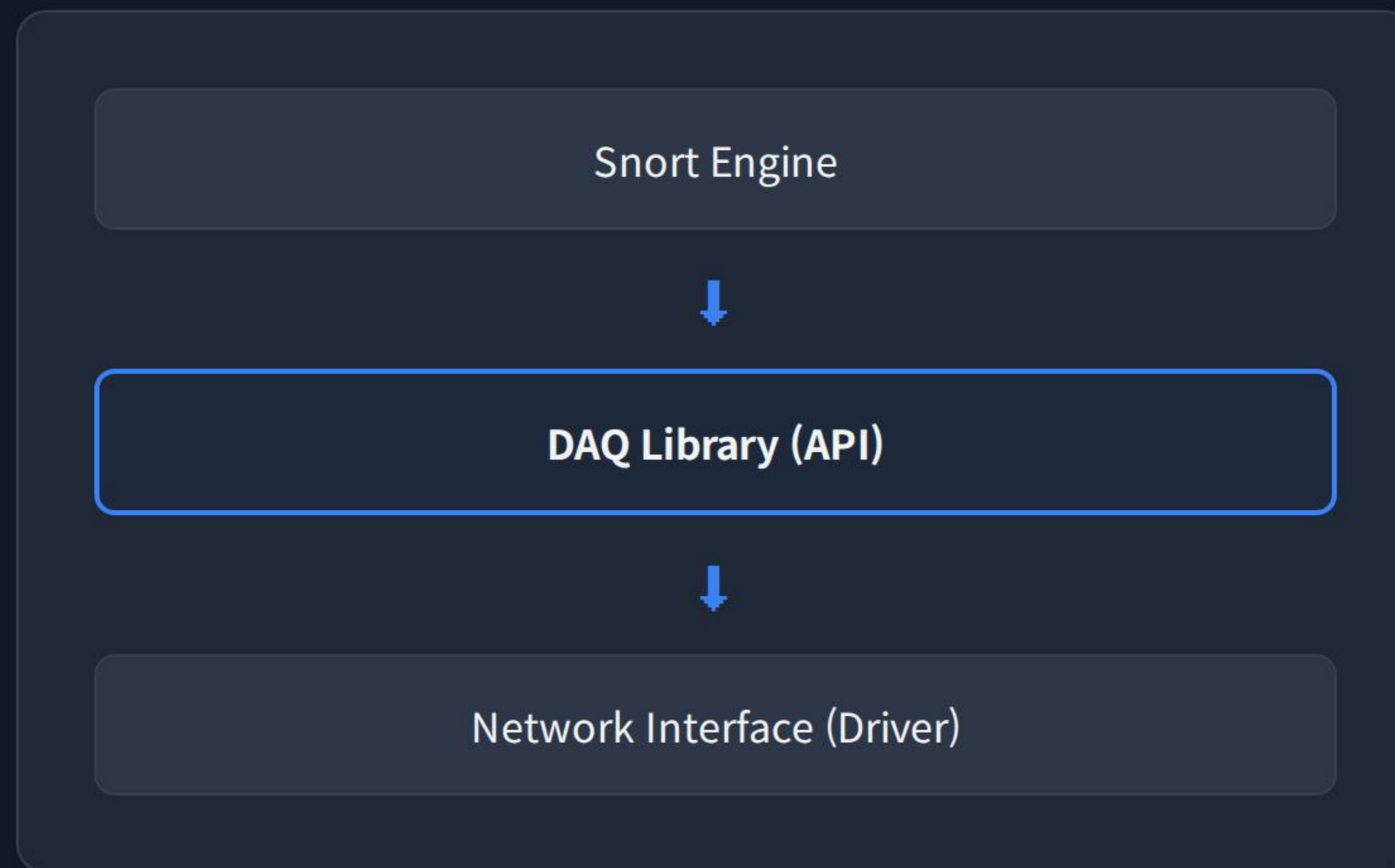
600 × 400

핵심 선결 조건: DAQ (Data Acquisition)

Snort와 하드웨어의 중계자

Snort는 네트워크 패킷을 직접 수집하지 않고, **DAQ 라이브러리**를 통해 수집합니다.

- **역할:** 하드웨어(NIC)와 Snort 엔진 사이의 추상화 계층 (Abstraction Layer).
- **문제점:** Snort 버전과 DAQ 버전은 **엄격하게 호환**되어야 합니다.
- **결론:** 소스 설치를 통해 Snort와 DAQ의 버전을 정확히 일치시켜 빌드해야만 런타임 오류(Runtime Error)를 막을 수 있습니다.



Linux Build Process 3단계

우리가 입력하는 세 가지 명령어는 각각 **설정(Config)** -> **변환(Build)** -> **배포(Deploy)**의 단계를 의미합니다.

1. ./configure

시스템 환경을 스캔하여
Makefile 생성

"재료 확인 및 설계도 작성"

2. make

소스 코드를 컴파일하여
바이너리(실행 파일) 생성

"실제 제작 및 조립"

3. make install

생성된 파일을
시스템 디렉토리로 복사

"최종 위치에 배치"

Step 1: ./configure (환경 분석)

`configure` 는 쉘 스크립트입니다. 이 단계의 핵심은 의존성 (Dependencies) 검사입니다.

- **헤더 파일 체크:** C언어 헤더(`.h`)가 시스템 경로(`/usr/include` 등)에 존재하는지 확인합니다.
- **라이브러리 체크:** 필요한 `.so` 또는 `.a` 파일이 있는지 링커를 통해 확인합니다.
- **결과물:** 시스템 환경 변수가 반영된 `Makefile` 이 생성됩니다. 이 파일이 없으면 다음 단계로 넘어갈 수 없습니다.

```
# 사용자가 옵션을 통해 커널 기능이나 경로를 직접 지정  
함  
$ ./configure --enable-sourcefire \  
  --with-daq-includes=/usr/local/include \  
  --with-daq-libraries=/usr/local/lib
```


Deep Dive: Makefile의 정체

Makefile은 컴파일러(GCC)에게 전달하는 **작업 지시서**입니다.

수천 개의 소스 파일 중 어떤 파일을 먼저 컴파일해야 하는지, 어떤 라이브러리와 연결(Link)해야 하는지 정의되어 있습니다.

의존성 그래프:

"A를 만들려면 B가 필요하고, B를 만들려면 C가 필요하다"는 관계가 명시되어 있어, 소스 코드가 수정되면 관련된 파일만 다시 컴파일하여 효율성을 높입니다.

600 × 400

Step 2: make (컴파일 및 링킹)

가장 시간이 오래 걸리는 단계로, 두 가지 핵심 작업이 일어납니다.

1. 컴파일 (Compilation):

C 소스 코드(`.c`)를 기계어인 오브젝트 파일(`.o`)로 변환합니다. 문법 오류가 있다면 여기서 중단됩니다.

2. 링킹 (Linking):

생성된 여러 오브젝트 파일들과 외부 라이브러리(DAQ, PCAP 등)를 하나로 묶어 최종 실행 파일(Executable Binary)을 만듭니다.

`-j` 옵션은 멀티코어를 사용하여 빌드 속도를 높임

```
$ make -j4
```

* gcc 명령어가 수없이 실행되며 `.o` 파일들을 생성하는 것을 볼 수 있습니다.

Step 3: make install (파일 배포)

빌드된 바이너리는 현재 폴더(build directory)에만 존재합니다. 이를 시스템 표준 경로로 이동시킵니다.

파일 종류	이동 경로 (Destination)	설명
실행 파일 (Binary)	<code>/usr/local/bin/snort</code>	사용자가 셸에서 명령어를 실행할 위치
설정 파일 (Config)	<code>/etc/snort</code> 또는 <code>/usr/local/etc</code>	snort.conf 등 설정 파일 위치
라이브러리 (Lib)	<code>/usr/local/lib/snort</code>	동적 플러그인 및 공유 라이브러리

⚠ 주의: 시스템 디렉토리에 쓰기 작업을 하므로 반드시 root 권한(sudo)이 필요합니다.

중요: Shared Library Cache (ldconfig)

"설치했는데 실행이 안 돼요!"

`make install` 로 라이브러리(`.so`)를 `/usr/local/lib` 에 복사했더라도, 운영체제는 이 새로운 파일의 존재를 모를 수 있습니다.

ldconfig 명령어의 역할

시스템 라이브러리 캐시(`/etc/ld.so.cache`)를 갱신하여, Snort 실행 시 OS가 필요한 라이브러리를 빠르게 찾을 수 있도록 인덱싱합니다.

```
$ snort -V
snort: error while loading shared libraries:
libdaq.so.2: cannot open shared object file
```

해결책: 라이브러리 캐시 갱신

```
$ sudo ldconfig
```


컴파일 옵션 전략 (Optimization)

소스 설치의 가장 큰 장점은 내 환경에 맞게 기능을 제어할 수 있다는 점입니다.

기능 활성화 (Features)

`--enable-sourcefire` : Snort의 핵심 성능 향상 기능 및 전용 룰셋 지원 활성화. (필수 권장)

`--enable-perfprofiling` : 어떤 전처리거나 룰이 CPU를 많이 쓰는지 분석하는 도구 포함.

경로 지정 (Paths)

`--prefix=/opt/snort` : 설치 위치를 변경하여 시스템 기본 경로와 분리 관리하고 싶을 때 사용. 삭제(Uninstall)가 용이함.

소스 설치 마스터 체크리스트

- ✓ **준비 (Prerequisite):**
컴파일러(gcc), 툴(make), 필수 라이브러리(libpcap, pcre, zlib, **libdaq-dev**) 설치 확인
- ⚙ **설정 (Configure):**
에러 없이 Makefile이 생성되었는가? (missing library 에러 체크)
- 🔨 **빌드 (Make):**
컴파일 도중 'Fatal Error' 없이 100% 완료되었는가?
- 🔗 **마무리 (Post-Install):**
`ldconfig` 실행 후 `snort -V` 로 버전 및 라이브러리 링크 확인

System Administrator's Perspective

"소스 설치는 단순한 복사 과정이 아니라,
시스템 환경에 맞춰 소프트웨어를 **통합(Integration)**하는 엔지니어링 과정입니다."

>_ Happy Hacking